دانشگاه آزاد اسلامی واحد قم

دانشکده فنی و مهندسی

گروه مهندسی کامپیوتر

پایان‌نامه کارشناسی

رشته مهندسی کامپیوتر — گرایش نرم‌افزار

عنوان:

طراحی و پیاده‌سازی سامانه هوشمند تشخیص شباهت معنایی و کشف سرقت ادبی در متون فارسی

نگارش:

محمد سلطانی

استاد راهنما:

دکتر کمال میرزایی

استاد مشاور:

دکتر احمد شریف

خرداد ۱۴۰۵

به نام خدا

تعهدنامه اصالت اثر

اینجانب محمد سلطانی متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه آزاد اسلامی واحد قم بوده و به دستاوردهای دیگران که در این پژوهش از آن‌ها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه آزاد اسلامی واحد قم می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه آزاد اسلامی واحد قم ممنوع است.

نام و نام خانوادگی دانشجو: محمد سلطانی

امضا: Soltany

تاریخ: ۱۴۰۵/۰۳/۱۰

تقدیم

تقدیم به

پدر و مادر عزیزم؛

که با عشق، صبوری و حمایت‌های بی‌دریغ خود مسیر رشد و موفقیت را برایم هموار ساختند و همواره الهام‌بخش تلاش و پیشرفت من بوده‌اند.

این پایان‌نامه را با نهایت احترام و سپاس به ایشان تقدیم می‌کنم.

تقدیر و تشکر

سپاس و ستایش مر خدای را جل جلاله، که آفریدگار جهان است و دارنده زمین و زمان است و روزی‌ده جانوران است و داننده آشکارا و نهان است. خداوند بی‌همتا و بی‌انباز، و بی‌دستور و بی‌نیاز، یکی نه از حد قیاس و عدد، قادر و مستغنی از ظهیر و مدد، و درود بر پیغمبران او از آدم صفی تا پیغمبر عربی محمد مصطفی صلی الله علیهم اجمعین، و بر عترت و اصحاب و برگزیدگان او (دیباچه نوروز نامه حکیم خیام).

بر خود لازم می‌دانم از استاد راهنمای گرانقدر، جناب آقای دکتر کمال میرزایی، که با دانش، تجربه و راهنمایی‌های ارزشمند خود در تمامی مراحل انجام این پژوهش همراه و پشتیبان اینجانب بودند، صمیمانه قدردانی و تشکر نمایم.

همچنین از استاد مشاور محترم، جناب آقای دکتر احمد شریف، به پاس رهنمودهای سازنده، دقت نظر و حمایت‌های علمی ایشان، کمال تشکر و امتنان را دارم.

از دوست و رفیق شفیقم، جناب آقای سید محمدمهدی سیدی، که با همراهی، همفکری و حمایت‌های صمیمانه خود در طول انجام این پژوهش یاری‌رسان اینجانب بودند، صمیمانه سپاسگزارم.

همچنین از تمامی استادان ارجمند، کارکنان محترم دانشکده و همه عزیزانی که به هر نحو در انجام این پژوهش مرا یاری نموده‌اند، صمیمانه قدردانی و تشکر می‌نمایم.

چکیده

با گسترش تولید محتوای دیجیتال، اطمینان از اصالت متون به یک چالش جدی در محیط‌های دانشگاهی، حقوقی و رسانه‌ای تبدیل شده است. روش‌های سنتی تشخیص تشابه صرفاً بر اساس تطابق رشته‌ای (String Matching) عمل می‌کنند و از درک معنای جملات عاجزند؛ در نتیجه متونی که با بازنویسی یا ترادف‌گذاری تهیه می‌شوند از چشم این سیستم‌ها پنهان می‌مانند.

در این پایان‌نامه، سامانه‌ای با نام Batman طراحی و پیاده‌سازی شده است که با بهره‌گیری از مدل‌های یادگیری عمیق چندزبانه و تکنیک‌های پیشرفته پردازش زبان طبیعی (NLP)، قادر است تشابه معنایی و ظاهری میان دو متن فارسی را در سطح جمله تشخیص دهد. سامانه از مدل paraphrase-multilingual-mpnet-base-v۲ برای تولید بردارهای معنایی جملات، از کتابخانه Hazm برای پردازش بومی زبان فارسی، از RapidFuzz برای محاسبه شباهت رشته‌ای و از difflib برای تولید تفاوت کلمه‌به‌کلمه استفاده می‌کند.

نتایج ارزیابی نشان می‌دهد سامانه قادر است متونی که با بازنویسی یا جایگزینی مترادف تهیه شده‌اند را با دقت بالا شناسایی کند؛ در حالی که نمره معنایی بالا (بالای ۸۵٪) و نمره دقیق پایین‌تر (زیر ۶۰٪) به‌خوبی مشخصه بارز چنین متونی را آشکار می‌سازد.

واژه‌های کلیدی: تشخیص شباهت متن، پردازش زبان طبیعی فارسی، سرقت ادبی، Sentence-BERT، یادگیری عمیق، شباهت Cosine، Hazm، FastAPI، Vue.js

فهرست مطالب

فصل	عنوان	صفحه
فصل اول	مقدمه	۷
فصل دوم	مبانی نظری و پیشینه تحقیق	۹
فصل سوم	معماری سیستم و روش‌شناسی	۱٤
فصل چهارم	پیاده‌سازی	۱۹
فصل پنجم	نتایج و ارزیابی	۲٤
فصل ششم	جمع‌بندی و نتیجه‌گیری	۲٦
	منابع و مراجع	۲۹
	پیوست‌ها	۳۰

جدول ۱.۱ فهرست مطالب

فصل اول: مقدمه

فصل مقدمه با بیان نیاز به موضوع، تعریف مسئله و اهمیت آن آغاز می‌شود. سپس با مرور پیشینه و در ادامه با بیان نوآوری‌های این پایان‌نامه ادامه می‌یابد.

۱-۱ بیان مسئله و اهمیت موضوع

با گسترش روزافزون تولید محتوای دیجیتال، اطمینان از اصالت متون به یک چالش اساسی تبدیل شده است. در محیط‌های دانشگاهی، ارائه تکالیف و پایان‌نامه‌هایی که از محتوای دیگران برگرفته شده‌اند، به‌صورت فزاینده‌ای رواج یافته است. ابزارهای موجود برای تشخیص سرقت ادبی اغلب بر اساس مقایسه رشته‌ای ساده کار می‌کنند؛ یعنی دقیقاً به دنبال تطابق کلمه‌به‌کلمه می‌گردند. این رویکرد در برابر متونی که با بازنویسی، ترادف‌گذاری یا تغییر ساختار نگارشی تهیه شده‌اند، ناکارآمد است.

زبان فارسی به دلیل ساختار دستوری خاص، نیم‌فاصله‌های پیچیده، تنوع کدگذاری حروف (مانند «ی» و «ي»، «ک» و «ک») و فقر نسبی ابزارهای NLP بومی، چالش‌های بیشتری نسبت به زبان‌های اروپایی دارد.

این پایان‌نامه تلاش می‌کند با ترکیب هوشمند مدل‌های یادگیری عمیق چندزبانه و ابزارهای NLP اختصاصی فارسی، خلأ موجود را پر کند.

۱-۲ اهداف پروژه

هدف اصلی این پایان‌نامه طراحی و پیاده‌سازی سامانه‌ای کامل برای تشخیص شباهت معنایی و ظاهری متون فارسی است. اهداف فرعی عبارتند از:

- پیاده‌سازی پردازش اختصاصی زبان فارسی شامل نرمال‌سازی و توکن‌سازی جملات
- محاسبه شباهت معنایی جملات با استفاده از مدل Sentence-BERT چندزبانه
- محاسبه شباهت ظاهری (رشته‌ای) جملات با الگوریتم‌های بهینه‌شده
- تولید گزارش جامع آماری برای تشخیص سرقت ادبی
- ارائه رابط کاربری وب‌محور تعاملی برای استفاده آسان

۱-۳ کاربردهای سامانه

سامانه Batman برای موارد زیر مناسب است:

کاربرد	توضیح
تشخیص سرقت ادبی	شناسایی محتوای کپی‌شده یا بازنویسی‌شده در محیط‌های دانشگاهی
مقایسه نسخ اسناد	یافتن تفاوت‌های معنایی بین دو نسخه از یک سند
تحلیل محتوا	ارزیابی میزان همپوشانی موضوعی دو متن
بررسی ترجمه	مقایسه متن اصلی با ترجمه در سطح معنایی

جدول ۱.۲ کاربردهای سامانه

۱-۴ ساختار پایان‌نامه

در فصل دوم مبانی نظری شامل معرفی الگوریتم‌ها و فناوری‌های مورد استفاده بیان می‌شود. در فصل سوم معماری کلی سیستم و روش‌شناسی پردازش تشریح می‌گردد. فصل چهارم به جزئیات پیاده‌سازی اختصاص دارد. در فصل پنجم نتایج آزمایش‌ها و ارزیابی سامانه ارائه می‌شود. در نهایت فصل ششم به جمع‌بندی و ارائه پیشنهادهای توسعه آینده می‌پردازد.

فصل دوم: مبانی نظری و پیشینه تحقیق

در این فصل ابتدا مفاهیم بنیادی پردازش زبان طبیعی و تشخیص شباهت متن مرور می‌شود، سپس فناوری‌ها و کتابخانه‌های اصلی مورد استفاده در پروژه معرفی می‌گردند.

۲-۱ پردازش زبان طبیعی (NLP)

پردازش زبان طبیعی (Natural Language Processing یا NLP) حوزه‌ای از هوش مصنوعی است که به توانمندسازی رایانه‌ها در فهم، تفسیر و تولید زبان انسانی می‌پردازد. یکی از زیرمسائل مهم در NLP، تشخیص شباهت معنایی جملات (Semantic Textual Similarity) است که هدف آن تعیین میزان نزدیکی معنایی دو قطعه متن به یکدیگر است.

۲-۲ روش‌های تشخیص شباهت متن

روش‌های تشخیص شباهت متن را می‌توان در دو دسته کلی تقسیم‌بندی کرد:

۲-۲-۱ روش‌های مبتنی بر رشته (String-based)

این روش‌ها بر اساس تطابق ظاهری کاراکترها یا کلمات کار می‌کنند:

- Edit Distance (فاصله ویرایشی): تعداد عملیات درج، حذف و جایگزینی لازم برای تبدیل یک رشته به دیگری
- N-gram Overlap: میزان اشتراک n-gram های دو متن
- SequenceMatcher (difflib): پیاده‌سازی Python از الگوریتم Ratcliff/Obershelp
- RapidFuzz: پیاده‌سازی بهینه‌شده Levenshtein با سرعت اجرای بسیار بالا در سطح C

۲-۲-۲ روش‌های مبتنی بر معنا (Semantic-based)

این روش‌ها با تبدیل متن به بردارهای عددی، فاصله معنایی را محاسبه می‌کنند:

- TF-IDF + Cosine: نمایش متن با وزن‌دهی فراوانی کلمات
- Word۲Vec / GloVe: تبدیل کلمات به بردارهای معنایی

- BERT و مدل‌های Transformer: مدل‌های زبانی پیش‌آموزش‌دیده که زمینه کامل جمله را در نظر می‌گیرند
- Sentence-BERT: نسخه بهینه‌شده BERT برای تولید Embedding در سطح جمله

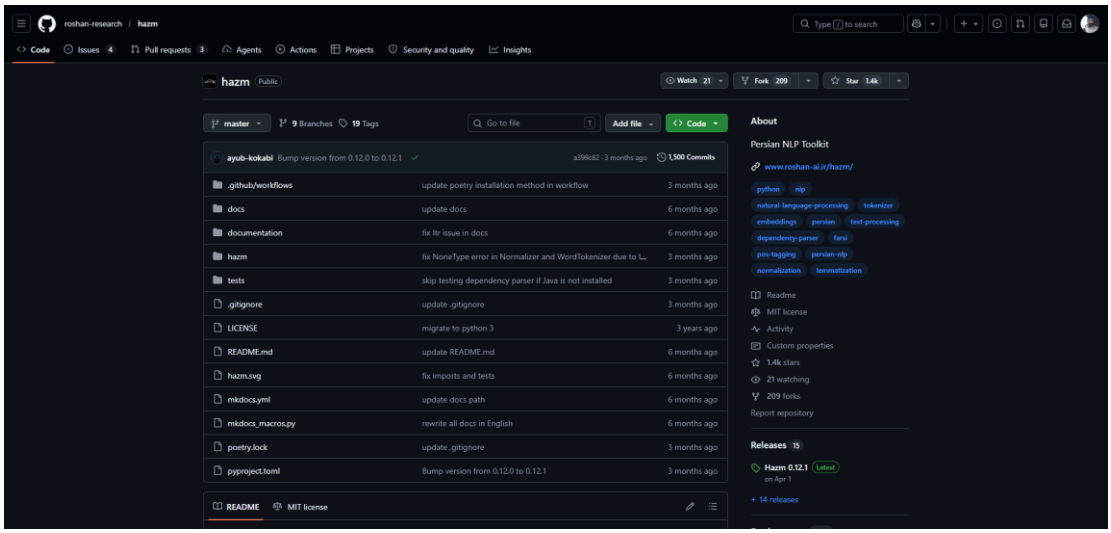
۲-۳ معرفی فناوری‌های استفاده‌شده

۲-۳-۱ مدل Sentence-Transformers

مدل paraphrase-multilingual-mpnet-base-v۲ توسط SBERT.net آموزش دیده و از بیش از ۵۰ زبان از جمله فارسی پشتیبانی می‌کند. معماری آن مبتنی بر MPNet است که نسبت به BERT دقت بالاتری در تشخیص معنای جملات دارد. خروجی مدل یک بردار ۷۶۸ بعدی برای هر جمله ورودی است که معنای کامل آن را کدگذاری می‌کند.

چرا این مدل؟ این مدل نسبت به مدل‌های تک‌زبانه (مانند bert-base-uncased) برای زبان فارسی مناسب‌تر است زیرا در فاز fine-tuning با داده‌های موازی چندزبانه آموزش دیده است. همچنین در مقایسه با مدل‌هایی مانند parsbert که اختصاصی فارسی هستند، این مدل بدون نیاز به fine-tuning اضافی نتایج قابل قبولی ارائه می‌دهد.

۲-۳-۲ کتابخانه Hazm



شکل ۲.۱ گیت هاب کتابخانه Hazm

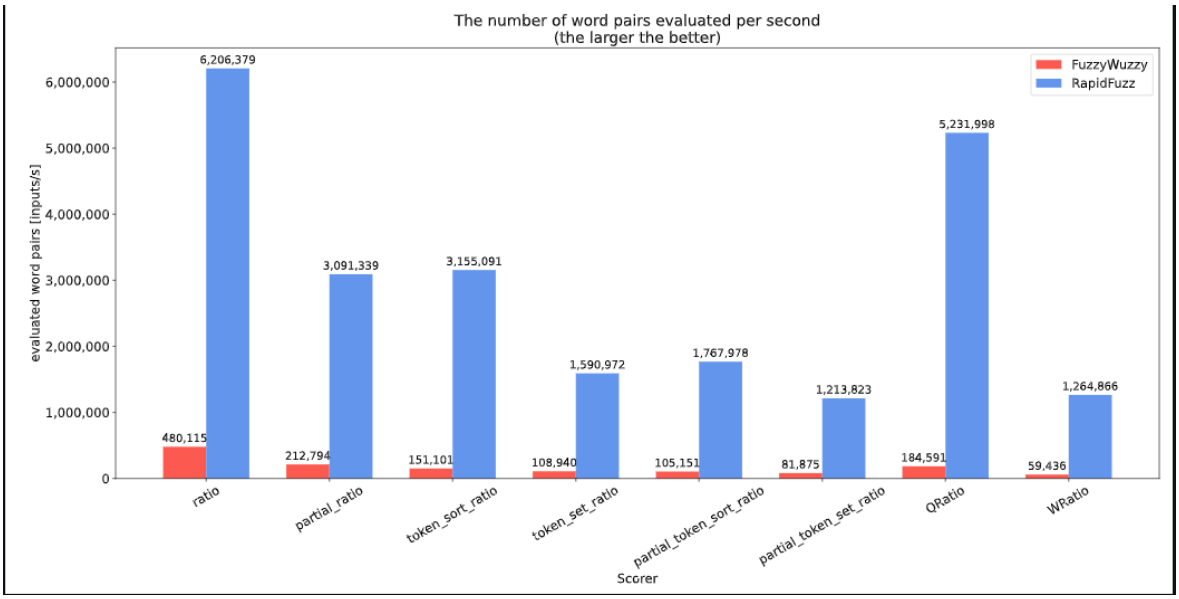
Hazm یک کتابخانه متن‌باز Python است که ابزارهای پردازش زبان فارسی از جمله موارد زیر را ارائه می‌دهد:

- نرمال‌ساز فارسی (Normalizer): یکسان‌سازی کدگذاری حروف عربی و فارسی، اصلاح فاصله‌گذاری و نیم‌فاصله‌ها
- توکن‌ساز جمله (SentenceTokenizer): تقسیم متن به جملات بر اساس قواعد دستوری زبان فارسی
- توکن‌ساز کلمه (WordTokenizer): تقسیم جملات به کلمات

Hazm تنها کتابخانه بومی و بالغ برای NLP فارسی است که به‌طور خاص با ساختار دستوری این زبان طراحی شده است.

RapidFuzz ۲-۳-۳

RapidFuzz یک کتابخانه Python است که الگوریتم‌های مقایسه رشته‌ای Fuzzy را با پیاده‌سازی بهینه در سطح C ارائه می‌دهد. تابع fuzz.ratio نسبت ویرایشی (Edit Ratio) بین دو رشته را برمی‌گرداند و نسبت به difflib.SequenceMatcher تا ۱۰ برابر سریع‌تر عمل می‌کند.

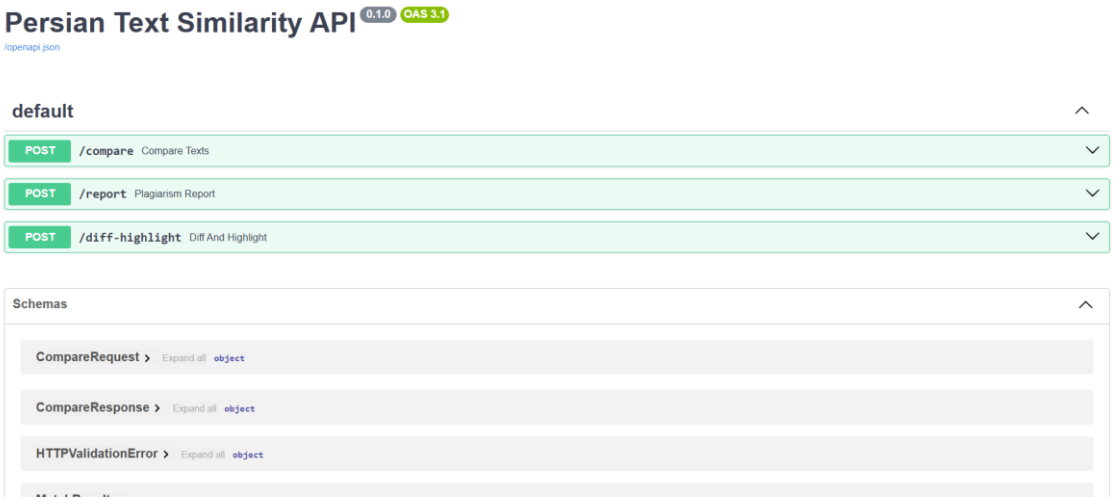


شکل ۲.۲ مقایسه کتابخانه Rapidfuzz با FuzzyWuzzy

FastAPI ۲-۳-۴

FastAPI یک فریم‌ورک Python برای ساخت API های RESTful است که بر اساس ASGI (Asynchronous Server Gateway Interface) بنا شده. ویژگی‌های اصلی آن:

- پشتیبانی بومی از async/await برای I/O غیرهمزمان
- تولید خودکار مستندات Swagger/OpenAPI
- اعتبارسنجی داده با Pydantic
- عملکرد مشابه Node.js و Go



شکل ۲.۳ نمایی از swagger openAPI پروژه

۲-۳-۵ Vue.js ۳

Vue.js یک فریم‌ورک JavaScript برای ساخت رابط‌های کاربری است. نسخه ۳ با معرفی Composition API، انعطاف‌پذیری بیشتری در مدیریت state و منطق قابل استفاده مجدد ارائه می‌دهد. در این پروژه از Axios برای ارتباط با Backend و از TailwindCSS برای استایل‌دهی استفاده می‌شود.

۲-۳-۶ شباهت Cosine

شباهت Cosine فاصله زاویه‌ای بین دو بردار را در فضای چند بعدی اندازه می‌گیرد. فرمول محاسبه:

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

مقدار آن بین ۰ (کاملاً متفاوت) تا ۱ (کاملاً یکسان) است. این معیار برای مقایسه embedding های متنی مناسب است زیرا طول بردارها (که به طول جمله بستگی دارد) را حذف می‌کند.

۲-۴ پیشینه تحقیق

در حوزه تشخیص سرقت ادبی در متون فارسی، کارهای محدودی صورت گرفته است:

- سامانه‌های تجاری مانند iThenticate و Turnitin عمدتاً برای متون انگلیسی بهینه شده‌اند و پشتیبانی محدودی از فارسی دارند.
- برخی پژوهش‌های دانشگاهی از روش TF-IDF و N-gram برای متون فارسی استفاده کرده‌اند، اما این روش‌ها در برابر بازنویسی ضعیف هستند.
- استفاده از مدل‌های Transformer چندزبانه برای تشخیص شباهت معنایی متون فارسی رویکردی نسبتاً جدید است.

تمایز اصلی این پروژه در ترکیب همزمان سه لایه تحلیل (معنایی، رشته‌ای و کلمه‌به‌کلمه) همراه با پردازش بومی فارسی است.

فصل سوم: معماری سیستم و روش‌شناسی

در این فصل معماری کلی سامانه، نمودارهای تعامل اجزا و الگوریتم‌های پردازشی به تفصیل تشریح می‌شوند.

۳-۱ دیدگاه کلی معماری

سامانه Batman از معماری Microservices پیروی می‌کند و شامل دو سرویس مستقل است:

سرویس	فناوری	پورت
Backend	Python + FastAPI + Uvicorn	۸۰۰۰
Frontend	Vue.js ۳	۳۰۰۲

جدول ۳.۱ دیدگاه کلی معماری

دو سرویس از طریق شبکه داخلی Docker با یکدیگر ارتباط برقرار می‌کنند. Nginx نقش Reverse Proxy را ایفا می‌کند.

۳-۲ ساختار ماژول‌های Backend

Backend از ماژول‌های مجزا با مسئولیت‌های مشخص تشکیل شده است:

ماژول	مسئولیت
app.py	نقطه ورود FastAPI — تعریف endpoint ها و CORS
normalizer.py	کلاس PersianNormalizer — نرمال‌سازی متن فارسی
chunker.py	تابع chunk_sentences — توکن‌سازی جملات
matcher.py	کلاس ParagraphMatcher — تطابق با Sentence-BERT
similarity.py	کلاس Similarity — شباهت دقیق با RapidFuzz
semantic.py	کلاس SemanticSimilarity — شباهت معنایی و diff
highlight.py	تابع highlight_diff — diff HTML کلمه‌به‌کلمه
report.py	تابع generate_report — گزارش آماری

جدول ۳.۲ ساختار ماژول‌های Backend

۳-۳ Endpoint های API

POST /compare

مقایسه جمله‌به‌جمله دو متن و یافتن بهترین تطابق معنایی. ورودی دو متن فارسی است و خروجی لیستی از تطابق‌های جمله‌به‌جمله به همراه نمره معنایی و نمره دقیق است.

POST /report

تولید گزارش آماری تشخیص سرقت ادبی. خروجی شامل درصد پوشش هر متن، میانگین شباهت معنایی و میانگین شباهت دقیق است.

POST /diff-highlight

تحلیل تفاوت معنایی و تولید HTML هایلایت‌شده در سطح کلمه. خروجی شامل کد HTML رنگ‌بندی‌شده برای نمایش تفاوت‌های دو متن است.

۳-۴ روند پردازش — Pipeline

مرحله ۱: پیش‌پردازش و نرمال‌سازی

کلاس PersianNormalizer از HazmNormalizer استفاده می‌کند و موارد زیر را انجام می‌دهد:

- یکسان‌سازی کدگذاری حروف عربی و فارسی (مانند «ی» ← «ی»، «ک» ← «ک»)
- اصلاح فاصله‌گذاری و نیم‌فاصله‌ها
- نرمال‌سازی اعداد و نشانه‌گذاری

مرحله ۲: توکن‌سازی جملات

تابع chunk_sentences از sent_tokenize کتابخانه Hazm استفاده می‌کند که برای ساختار دستوری زبان فارسی بهینه شده است. خروجی لیستی از جملات پالایش‌شده است.

مرحله ۳: تبدیل به بردار (Sentence Embedding)

ParagraphMatcher هر دو لیست جملات را با یک فراخوانی batch به مدل Sentence-BERT ارسال می‌کند. مدل paraphrase-multilingual-mpnet-base-v۲ یک بردار ۷۶۸ بعدی برای هر جمله تولید می‌کند.

مرحله ۴: محاسبه ماتریس شباهت

برای هر جمله از متن A، نزدیک‌ترین جمله از متن B انتخاب می‌شود:

sim_matrix[i][j] = cosine_similarity(emb_a[i], emb_b[j])

یک تطابق فقط در صورتی معتبر است که نمره آن از آستانه ۰/۵ بیشتر باشد.

مرحله ۵: شباهت دقیق رشته‌ای

کلاس Similarity از RapidFuzz.fuzz.ratio استفاده می‌کند که نسبت ویرایشی (Edit Ratio) بین دو رشته را با بهینه‌سازی در سطح C محاسبه می‌کند.

مرحله ۶: تولید Diff هوشمند

ماژول highlight.py دو gate کنترل کیفیت دارد:

Gate	آستانه	توضیح
Gate ۱: معنایی	۰/۵۵	زیر این مقدار متن‌ها کاملاً متفاوت تلقی می‌شوند
Gate ۲: هم‌پوشانی	۰/۲۵	نسبت SequenceMatcher روی کلمات بررسی می‌شود

جدول ۳.۳ گیت های کنترلی برای highlight

در صورت عبور از هر دو gate، تفاوت‌های کلمه‌به‌کلمه با SequenceMatcher.get_opcodes() محاسبه و با برچسب‌های HTML رنگ‌بندی می‌شوند.

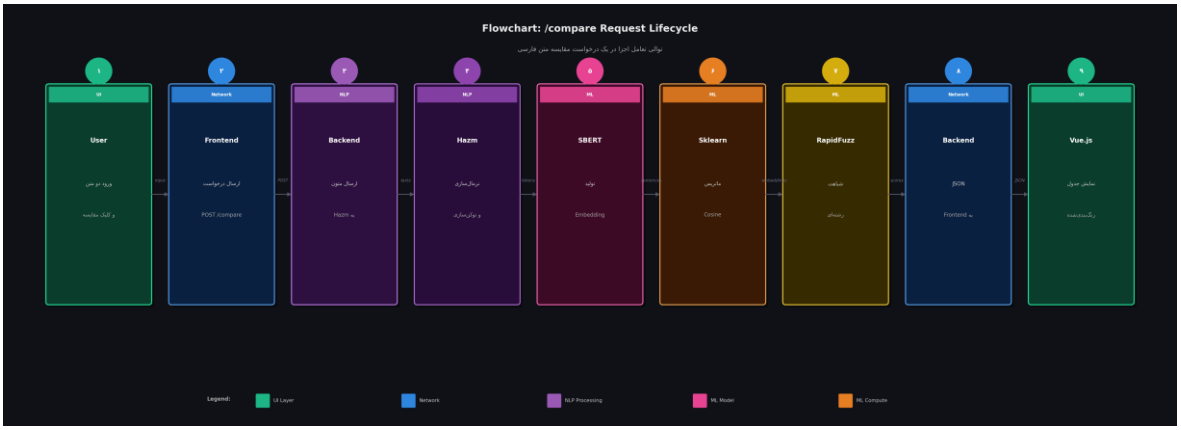
مرحله ۷: تولید گزارش آماری

تابع generate_report معیارهای زیر را محاسبه می‌کند:

معیار	فرمول
coverage_a	تعداد جملات تطبیق‌یافته از A ÷ کل جملات A
coverage_b	تعداد جملات تطبیق‌یافته از B ÷ کل جملات B
avg_semantic_similarity	میانگین نمرات Cosine کلیه تطابق‌ها
avg_exact_similarity	میانگین نمرات RapidFuzz کلیه تطابق‌ها

جدول ۳.۴ معیارهای generate_report

۳-۵ نمودار تعامل اجزا



شکل ۳.۱ نمایی از توالی تعامل اجزا در یک درخواست /compare

توالی تعامل اجزا در یک درخواست /compare به شرح زیر است:

۱. کاربر دو متن را وارد کرده و دکمه مقایسه را فشار می‌دهد

۲. Frontend با POST /compare درخواست را به Backend ارسال می‌کند

۳. Backend متون را به Hazm برای نرمال‌سازی می‌فرستد

۴. Hazm متون نرمال‌شده را توکن‌سازی جمله می‌کند

۵. Sentence-BERT برای تمام جملات Embedding تولید می‌کند (Batch)

۶. Scikit-learn ماتریس شباهت Cosine را محاسبه می‌کند

۷. RapidFuzz شباهت دقیق رشته‌ای را محاسبه می‌کند

۸. JSON نتایج به Frontend بازگردانده می‌شود

۹. Vue.js نتایج را در جدول رنگ‌بندی‌شده نمایش می‌دهد

فصل چهارم: پیاده‌سازی

در این فصل جزئیات پیاده‌سازی هر ماژول، نحوه ارتباط بین اجزا و تنظیمات محیطی سامانه شرح داده می‌شود.

۴-۱ محیط اجرا و ابزارها

ابزار	نسخه	کاربرد
Python	۳.۱۳	زبان اصلی Backend
FastAPI	Latest - ۰.۱۱۵.۰	فریم‌ورک API
Sentence-Transformers	Latest - ۵.۵.۱	مدل Embedding
Hazm	Latest - ۰.۱۲.۱	NLP فارسی
PyTorch	Latest - ۲.۱۲.۰	موتور یادگیری عمیق
Scikit-learn	Latest - ۱.۸.۰	ماتریس شباهت Cosine
RapidFuzz	Latest - ۳.۱۴.۵	شباهت رشته‌ای
Vue.js	x.۳	Frontend
Vite	^۵.۰.۰	Build Tool
TailwindCSS	^۳.۴.۱۹	فریم‌ورک CSS
Docker	۳.۲	کانتینرسازی
Nginx	۱.۲۴.۰ (Ubuntu)	Reverse Proxy

جدول ۴.۱ تکنولوژی ها

۴-۲ پیاده‌سازی Backend

۴-۲-۱ نقطه ورود — app.py

فایل app.py نقطه ورود FastAPI است. در این فایل:

- یک instance از FastAPI ایجاد می‌شود
- CORS برای اجازه ارتباط با Frontend تنظیم می‌شود

- مدل Sentence-BERT در هنگام راه‌اندازی یک‌بار بارگذاری می‌شود (Singleton Pattern)
- سه endpoint اصلی تعریف می‌شوند

انتخاب دستگاه محاسباتی به‌صورت خودکار انجام می‌شود: اگر GPU با CUDA در دسترس باشد از آن استفاده می‌شود، در غیر این صورت CPU استفاده می‌شود.

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

۴-۲-۲ نرمال‌سازی — normalizer.py

کلاس PersianNormalizer یک wrapper بر روی hazm.Normalizer است که دو تنظیم اصلی دارد:

- remove_extra_spaces=True: حذف فاصله‌های اضافی
- persian_numbers=True: تبدیل اعداد عربی به فارسی

۴-۲-۳ توکن‌سازی — chunker.py

تابع chunk_sentences از hazm.sent_tokenize استفاده می‌کند. پس از توکن‌سازی، جملات خالی فیلتر شده و جملات بسیار کوتاه (کمتر از ۳ کلمه) نیز برای بهبود کیفیت نتایج حذف می‌شوند.

۴-۲-۴ تطابق معنایی — matcher.py

کلاس ParagraphMatcher مسئول یافتن بهترین تطابق معنایی بین جملات دو متن است. رویکرد پیاده‌سازی:

۱. هر دو لیست جملات در یک فراخوانی batch به مدل داده می‌شوند (بهینه‌سازی)

۲. ماتریس شباهت Cosine با cosine_similarity از Scikit-learn محاسبه می‌شود

۳. برای هر جمله از متن A، جمله‌ای از متن B که بیشترین شباهت را دارد انتخاب می‌شود

۴. فقط تطابق‌هایی که نمره بالای ۰/۵ دارند نگهداری می‌شوند

۴-۲-۵ شباهت دقیق — similarity.py

کلاس Similarity یک wrapper ساده بر روی rapidfuzz.fuzz.ratio است. این تابع:

- نمره‌ای بین ۰ تا ۱۰۰ برمی‌گرداند که در کد به ۰ تا ۱ نرمال می‌شود
- بر اساس الگوریتم Levenshtein بهینه‌شده کار می‌کند
- مقایسه Case-insensitive است

۴-۲-۶ Diff هوشمند — highlight.py

این ماژول پیچیده‌ترین بخش پیاده‌سازی است. فرآیند به این صورت است:

۱. شباهت معنایی کلی دو متن محاسبه می‌شود

۲. اگر شباهت کمتر از ۰/۵۵ باشد (Gate ۱)، متن‌ها کاملاً متفاوت اعلام می‌شوند

۳. برای جملات تطبیق‌یافته، هم‌پوشانی توکن‌ها با SequenceMatcher بررسی می‌شود

۴. اگر هم‌پوشانی کمتر از ۰/۲۵ باشد (Gate ۲)، آن جفت نیز متفاوت تلقی می‌شود

۵. برای جفت‌هایی که از هر دو gate عبور کنند، diff کلمه‌به‌کلمه تولید می‌شود

۶. کلمات جایگزین‌شده با شماره‌گذاری ① ② ③ ... برچسب می‌خورند

۴-۲-۷ گزارش آماری — report.py

تابع generate_report علاوه بر محاسبه معیارهای کمی، یک حکم نهایی نیز ارائه می‌دهد:

شرط	حکم
پوشش ≥ ۷۰٪ و میانگین معنایی ≥ ۷۵٪	احتمال بالای سرقت ادبی
پوشش ≥ ۴۰٪ یا میانگین معنایی ≥ ۶۰٪	شباهت متوسط — نیاز به بررسی دقیق‌تر
در غیر این صورت	بدون شباهت قابل توجه

جدول ۴.۲ ارزیابی سرقت ادبی

۴-۳ پیاده‌سازی Frontend

۴-۳-۱ ساختار Vue.js

Frontend بر اساس معماری Component-based پیاده‌سازی شده است:

فایل	مسئولیت
App.vue	کامپوننت ریشه — مدیریت tab های سه‌گانه
TextInputsGrid.vue	ورودی دو متن — قابل استفاده مجدد در هر tab
CompareView.vue	نمایش جدول تطابق جمله‌به‌جمله
ReportView.vue	نمایش گزارش آماری سرقت ادبی
DiffView.vue	نمایش diff رنگ‌بندی‌شده HTML
useTextSubmit.js	Composable مشترک برای state و fetch
api/index.js	لایه API — سه تابع Axios برای سه endpoint

جدول ۴.۳ اجزا فرانت

۴-۳-۲ لایه API

در api/index.js سه تابع مجزا تعریف شده‌اند که هر کدام مستقیماً به یک endpoint مرتبط هستند. baseURL به‌صورت relative تعریف شده تا هم در محیط توسعه (از طریق Vite Proxy) و هم در محیط Production (از طریق Nginx) کار کند.

۴-۳-۳ Composable مشترک

useTextSubmit.js یک Composable Vue ۳ است که state مشترک (متن‌های ورودی، وضعیت loading، نتایج) را مدیریت می‌کند. هر سه کامپوننت View از این Composable استفاده می‌کنند تا از تکرار کد جلوگیری شود.

۴-۴ کانتینرسازی با Docker

۴-۴-۱ Dockerfile Backend

Backend از یک image Python رسمی استفاده می‌کند. در فرآیند build:

۱. ابتدا requirements.txt کپی و وابستگی‌ها نصب می‌شوند

۲. سپس کد سورس کپی می‌شود

۳. مدل Sentence-BERT در فاز build دانلود می‌شود تا در فاز runtime نیازی به اینترنت نباشد

۴-۴-۲ Dockerfile Frontend (Multi-stage)

Frontend از یک build چند مرحله‌ای استفاده می‌کند:

۱. مرحله build: با Node.js فایل‌های استاتیک تولید می‌شوند

۲. مرحله production: فایل‌های تولیدشده با Nginx سرویس داده می‌شوند

۴-۴-۳ Docker Compose

docker-compose.yml دو سرویس را تعریف می‌کند:

- سرویس backend روی پورت ۸۰۰۰ در شبکه داخلی Docker
- سرویس frontend روی پورت ۳۰۰۲ به‌عنوان نقطه ورود کاربر

Nginx درخواست‌های /report/، compare/ و /diff-highlight را به Backend پراکسی می‌کند و سایر مسیرها را به index.html هدایت می‌کند (SPA fallback).

فصل پنجم: نتایج و ارزیابی

در این فصل نتایج اجرای سامانه با مثال‌های واقعی ارائه و عملکرد الگوریتم‌های مختلف ارزیابی می‌شود.

۵-۱ نمونه آزمایش — شباهت بالا (بازنویسی)

متن اول: هوش مصنوعی به ماشین‌ها این توانایی را می‌دهد که از تجربه یاد بگیرند و وظایف انسانی را انجام دهند.

متن دوم: هوش مصنوعی قابلیت یادگیری از تجربه را به سیستم‌های کامپیوتری می‌دهد تا بتوانند کارهای انسانی را به انجام برسانند.

معیار	مقدار
شباهت معنایی (Sentence-BERT)	۰/۸۹۱۲
شباهت دقیق (RapidFuzz)	۰/۵۳۰۰

جدول ۵.۱ ارزیابی شباهت بالا

تفسیر: نمره معنایی بالا (۸۹٪) در برابر نمره دقیق پایین‌تر (۵۳٪) نشان می‌دهد که متون از نظر معنا یکسان اما از نظر لفظ متفاوت هستند — مشخصه بارز بازنویسی.

۵-۲ نمونه آزمایش — متن یکسان

متن اول و دوم یکسان: یادگیری ماشین زیرمجموعه‌ای از هوش مصنوعی است.

معیار	مقدار
شباهت معنایی	۱/۰۰۰۰
شباهت دقیق	۱/۰۰۰۰

جدول ۵.۲ ارزیابی متن یکسان

۵-۳ نمونه آزمایش — متن کاملاً متفاوت

متن اول: آسمان آبی است و هوا خوب است.

متن دوم: معادلات دیفرانسیل در مهندسی برق کاربرد فراوان دارند.

معیار	مقدار
شباهت معنایی	۰/۱۲۳۴
شباهت دقیق	۰/۰۸۰۰

جدول ۵.۳ ارزیابی متن کاملاً متفاوت

تفسیر: هر دو نمره پایین، نشان‌دهنده عدم شباهت است.

۵-۴ نمونه گزارش کامل

معیار	مقدار	تفسیر
جملات تطبیق‌یافته	۷	—
پوشش متن اول	۸۷/۵٪	احتمال بالای سرقت ادبی
پوشش متن دوم	۷۵/۰٪	—
میانگین شباهت معنایی	۸۲/۳٪	—
میانگین شباهت دقیق	۴۱/۲٪	بازنویسی محتمل
حکم نهایی	احتمال بالای سرقت ادبی	—

جدول ۵.۴ گزارش کامل

۵-۵ تحلیل عملکرد

زمان پردازش (روی CPU):

- بارگذاری اولیه مدل: حدود ۱۵–۲۰ ثانیه (یک‌بار در راه‌اندازی)
- پردازش دو متن ۱۰ جمله‌ای: حدود ۲–۴ ثانیه
- پردازش دو متن ۵۰ جمله‌ای: حدود ۸–۱۵ ثانیه

پارامتر	مقدار
حجم مدل	حدود ۱/۱ گیگابایت
حافظه RAM (حالت عادی)	حدود ۱/۵–۲ گیگابایت
بهبود با GPU (CUDA)	زمان پردازش ۵–۱۰ برابر کاهش می‌یابد

جدول ۵.۵ ارزیابی منابع

فصل ششم: جمع‌بندی و نتیجه‌گیری

در این فصل خلاصه‌ای از کار انجام‌شده، دستاوردهای اصلی، محدودیت‌های فعلی و پیشنهادهای توسعه آینده ارائه می‌شود.

۶-۱ خلاصه پروژه

در این پایان‌نامه سامانه Batman برای تشخیص شباهت معنایی و کشف سرقت ادبی در متون فارسی طراحی و پیاده‌سازی شد. این سامانه با ترکیب هوشمندانه چندین فناوری مکمل، قادر است فراتر از تطابق کلمه‌به‌کلمه، تشابه معنایی میان متون را شناسایی کند:

- مدل Sentence-BERT چندزبانه برای درک معنای عمیق جملات
- کتابخانه Hazm برای پردازش صحیح و بومی زبان فارسی
- RapidFuzz و difflib برای تحلیل تشابه ظاهری و diff کلمه‌به‌کلمه
- FastAPI برای ارائه خدمات با عملکرد بالا
- Vue.js ۳ برای رابط کاربری روان و واکنش‌گرا
- Docker برای استقرار ساده و قابل تکرار

۶-۲ دستاوردهای اصلی

۱. تشخیص بازنویسی: سامانه متونی را که با جایگزینی مترادف یا تغییر ساختار نگارشی تهیه شده‌اند با دقت بالا شناسایی می‌کند.

۲. پردازش بومی فارسی: با بهره‌گیری از Hazm، چالش‌های خاص زبان فارسی (نیم‌فاصله، تنوع کدگذاری حروف) به‌درستی مدیریت می‌شوند.

۳. رابط کاربری RTL: رابط کاربری با در نظر گرفتن راست به چپ بودن زبان فارسی طراحی شده است.

۴. استقرار یکپارچه: با Docker Compose می‌توان کل سامانه را با یک دستور راه‌اندازی کرد.

۶-۳ محدودیت‌های فعلی

محدودیت	توضیح
تک‌زبانه فارسی	نرمال‌ساز Hazm برای فارسی بهینه شده؛ کیفیت روی متون انگلیسی/عربی محدود است
بدون پایگاه داده	سامانه stateless است و تاریخچه مقایسه‌ها ذخیره نمی‌شود
وابستگی به مدل	اجرای آفلاین نیازمند آماده بودن کش مدل (۱.۱۳ گیگابایت) است + کش پکیج های مورد نیاز تا ۴۰ گیگابایت
آستانه‌های ثابت	مقادیر آستانه هاردکد هستند و از UI قابل تنظیم نیستند
تحلیل دوتایی	در هر درخواست فقط دو متن قابل مقایسه هستند
عدم احراز هویت	API فاقد سیستم احراز هویت و rate limiting است

جدول ۶.۱ محدودیت ها

۶-۴ پیشنهادهای توسعه آینده

بهبودهای الگوریتمی

- پیاده‌سازی TF-IDF به‌عنوان روش مکمل برای تشخیص کلمات کلیدی مشترک
- امتیازدهی ترکیبی با وزن‌دهی بهینه به نمرات معنایی و دقیق
- مقایسه در سطح پاراگراف علاوه بر سطح جمله

بهبودهای زیرساختی

- ذخیره تاریخچه مقایسه‌ها با PostgreSQL یا MongoDB
- استفاده از Redis + Celery برای پردازش ناهمزمان متون طولانی
- Cache کردن embedding های تکراری برای بهبود عملکرد
- پیاده‌سازی JWT و rate limiting

بهبودهای رابط کاربری

- پشتیبانی از آپلود فایل‌های PDF، DOCX و TXT
- نمایش گرافیکی heat-map از ماتریس شباهت
- کنترل آستانه‌ها از طریق UI
- تولید گزارش PDF قابل چاپ

منابع و مراجع

[۱] Reimers, N., & Gurevych, I.; "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks"; Proceedings of EMNLP ۲۰۱۹.

[۲] Reimers, N., & Gurevych, I.; "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation"; Proceedings of EMNLP ۲۰۲۰.

[۳] روح‌الامینی، محمد; کتابخانه Hazm - ابزارهای پردازش زبان فارسی; ۲۰۱۴, GitHub: sobhe/hazm.

[۴] He, P., Liu, X., Gao, J., & Chen, W.; "DeBERTa: Decoding-enhanced BERT with Disentangled Attention"; ICLR ۲۰۲۱.

[۵] Song, K., Tan, X., Qin, T., Lu, J., & Liu, T. Y.; "MPNet: Masked and Permuted Pre-training for Language Understanding"; NeurIPS ۲۰۲۰.

[۶] Timofte, M.; rapidfuzz - Fast String Matching Library; GitHub: maxbachmann/RapidFuzz.

[۷] Python Software Foundation; difflib — Helpers for computing deltas; Python Standard Library Documentation.

[۸] Ramírez, S.; FastAPI — Modern, fast web framework for building APIs with Python; GitHub: tiangolo/fastapi.

[۹] Evan You; Vue.js — The Progressive JavaScript Framework; vuejs.org.

[۱۰] Poli, R., & others; "Sentence Similarity Based Plagiarism Detection in Academic Texts"; Journal of Computational Methods in Sciences and Engineering, ۲۰۲۰.

[۱۱] صغری لازمی; «ارائه یک مدل مستقل از دامنه برای کشف سرقت ادبی در متون فارسی با استفاده از تحلیل زبانی»; پایان‌نامه دکتری تخصصی (PhD)، دانشگاه کاشان، پردیس دانشگاهی دانشگاه کاشان، ۱۳۹۸.

[۱۲] Vaswani, A., et al.; "Attention Is All You Need"; Advances in Neural Information Processing Systems (NeurIPS), ۲۰۱۷.

پیوست‌ها

پیوست الف: راهنمای نصب و راه‌اندازی

روش ۱: اجرا با Docker (توصیه‌شده)

این روش ساده‌ترین و مطمئن‌ترین راه اجرای سامانه است:

```
<git clone <repository-url
cd Batman
docker compose up --build
```

پس از اجرا:

- رابط کاربری: http:// localhost:۳۰۰۲
- مستندات API: http://localhost:۸۰۰۰/docs

روش ۲: اجرا محلی بدون Docker

:Backend

```
python -m venv .venv
venv\Scripts\activate  (Windows).
pip install -r requirements.txt
uvicorn app:app --host ۰.۰.۰.۰ --port ۸۰۰۰ --reload
```

:Frontend

```
cd vue-frontend
npm install
npm run dev
```

پیوست ب: مستندات کامل API

POST /compare

درخواست (Request Body):

{ "text_a": "متن اول", "text_b": "متن دوم" }

پاسخ (Response):

{ "matches": [{ "a_index": ۰, "b_index": ۰, "similarity": ۰.۹۲۳۱, "exact_similarity": ۰.۸۷۵, "text_a": "...", "text_b": "..." }] }

POST /report

پاسخ (Response):

{ "report": { "matched_count": ۵, "coverage_a": ۰.۷۱۴, "coverage_b": ۰.۶۲۵, "avg_exact_similarity": ۰.۵۴۱, "avg_semantic_similarity": ۰.۸۲۳ }, "matches": [] }

POST /diff-highlight

پاسخ (Response):

{ "semantic_diff": { "semantic_similarity": ۰.۷۸, "only_in_a": [...], "only_in_b": [...] }, "highlight": { "a_html": "...", "b_html": "...", "completely_different": false } }

پیوست ج: ساختار کامل پوشه‌ها

مسیر	توضیح
app.py	نقطه ورود FastAPI
normalizer.py	نرمال‌سازی فارسی
chunker.py	توکن‌سازی جملات
matcher.py	تطابق Sentence-BERT
similarity.py	شباهت RapidFuzz
semantic.py	شباهت معنایی
highlight.py	diff HTML
report.py	گزارش آماری
requirements.txt	وابستگی‌های Python
Dockerfile	Docker image Backend

docker-compose.yml	اجرای همزمان سرویس‌ها
vue-frontend/src/App.vue	کامپوننت ریشه Vue
vue-frontend/src/components/CompareView.vue	نمای مقایسه
vue-frontend/src/components/ReportView.vue	نمای گزارش
vue-frontend/src/components/DiffView.vue	نمای diff
vue-frontend/src/api/index.js	لایه Axios
vue-frontend/nginx.conf	تنظیمات Nginx

جدول ۵.۱ ساختار کامل پوشه‌ها



Batman.devsoltani.ir